Python Programming: Main Method, Classes, and Objects

1 Theoretical Overview: Classes and Objects

In Python, almost everything is an object. To understand Objects, we must first understand Classes.

- **What is a Class?**
  A class is a blueprint or a template for creating objects. It defines a set of attributes (data) and methods (behavior) that the created objects will have.
  Example: A 'Car' class might define attributes like 'color' and 'model' and methods like 'drive()' or 'brake()'.

- **What is an Object?**
  An object is an instance of a class. When a class is defined, no memory is allocated until an object of that class is created.
  Example: If 'Car' is the class, 'my_red_tesla' is an object.

- **The __init__ Method**
  Commonly called the 'constructor', this special method is automatically called when a new object is initiated. It is used to initialize the object's state (attributes).

- **The 'self' Parameter**
  The 'self' parameter is a reference to the current instance of the class. It is used to access variables and methods associated with that specific object. It must be the first parameter of any method in a class.

2 The Python 'Main' Method Pattern

The following code demonstrates the standard pattern for creating an entry point in Python using the `if __name__ == "__main__":` idiom.

```
def process_data(data):
    """A sample processing function."""
    return [item.upper() for item in data]

def main():
    """Main function for primary logic."""
    print("--- Example Execution ---")
    print("Execution started...")

    sample_items = ["python", "main", "method", "example"]
    result = process_data(sample_items)

    print(f"Processed results: {result}")
    print("Execution finished successfully.\n")

```

```python
if __name__ == "__main__":
    main()
```

Listing 1: Main Method Implementation

3 Practice Programs

Below are three practice challenges implemented within the modular structure.

3.1 Practice 1: Simple Calculator

```python
def practice_1_calculator():
    def add(a, b): return a + b

    print("Practice 1: Simple Addition")
    num1, num2 = 10, 5
    print(f"{num1} + {num2} = {add(num1, num2)}\n")
```

3.2 Practice 2: Filename Utility

```python
def practice_2_file_checker():
    def is_python_file(filename):
        return filename.endswith('.py')

    print("Practice 2: Filename Utility")
    files = ["script.py", "data.csv", "main.py"]
    for f in files:
        status = "Python File" if is_python_file(f) else "Other"
        print(f"{f}: {status}")
```

3.3 Practice 3: List Statistics

```python
def practice_3_list_stats():
    def get_stats(numbers):
        if not numbers: return None
        return {"min": min(numbers), "max": max(numbers)}

    print("Practice 3: List Stats")
    nums = [23, 89, 12, 55, 4]
    print(f"Stats for {nums}: {get_stats(nums)}\n")
```